

(Preview of the progress)

0- Reproduction of what simone did.

1- Reset by subtract (soft-reset),

- a. Added soft-reset with recurrent
 - i. for this I used Simone's script
- b. Added soft-reset without the recurrent,
 - i. for this I trained the model by myself,
 - ii. Implementation (subtract at the next step)
 - iii. (Metadata need to be added, this way its decided, if its soft or hard reset.

2-Convolution support

- a. Trained the model by following the ST-MNIST tutorial in the snntorch.
- b. Studied the conv mechanism
- c. Basic implementation is done (inherently optimized because of the quantized weights)
- d. Tested, with debug prints
- e. Created graphs

3- MCU deployment.

- a. First CMSIS-NN and CMSIS-DPS libraries added to the project.
- b. UART DMA, needed for a big dataset while testing Simone's tests bcs of the difference in the FLASH size.
 - i. UART-DMA has been setup, and python script created to send the test data.
- c. Quad-SPI flash,
 - i. When started with the spiking CNN, the flash was not even enough for the weights.
 - ii. So QSPI set up to store the weights and the test data.

1- Simone's steps reproduced. (with the current snn2mcu script)

```
cd /home/emre/AllProjects/work_done_presentation/simones-test
```

```
python3 SNN Torch To NIR.py
```

```
python3 nir_to_c_generator_V2_commit_25may.py snntorch_braille7_model.nir
```

(after the files are generated, we need to put them in the project)

```
rm ./for-soft-reset/Core/Src/lif_neuron_gen.c
```

```
rm ./for-soft-reset/Core/Inc/lif_neuron_gen.h
```

```
cp lif_neuron_gen.c ./for-soft-reset/Core/Src/
```

```
cp lif_neuron_gen.h ./for-soft-reset/Core/Inc/
```

```
gnome-terminal
```

```
cd for-soft-reset/
```

```
code .
```

(Make sure the correct UART is chosen, for STM32H750 and in my project the UART3 is configured, so change the UART accordingly in the lif_neuron_gen.c

```
( HAL_UART_Transmit(&huart3, (uint8_t*)str, strlen(str), 1000); )
```

(make sure the board is connected)

(bottom left corner, BUILD)

(left side, RUN AND DEBUG)

(choose, ST-link gdb server)

(Wait for it to flash)

(Open a new terminal)

ti0 /dev/ttyACM0

(maybe the name is different check it in the /dev folder)

(now, press continue on top or f5)

(look at the tio terminal output)

Should be: (Right side, Simone's results from the thesis. Results are aligned)

```
THE SIMULATION BEGINS...
Output spikes accumulation: 0 49 28 23 18 15 21
Predicted class: 1
Expected class: 1
Match
Output spikes accumulation: 0 9 26 31 30 9 5
Predicted class: 3
Expected class: 3
Match
Output spikes accumulation: 0 29 34 40 30 9 18
Predicted class: 3
Expected class: 2
Miss
Output spikes accumulation: 0 38 39 22 21 26 18
Predicted class: 2
Expected class: 2
Match
Output spikes accumulation: 5 16 8 8 16 19 12
Predicted class: 5
Expected class: 6
Miss
Output spikes accumulation: 0 56 19 6 16 19 27
Predicted class: 1
Expected class: 1
Match
Output spikes accumulation: 0 47 23 22 27 15 25
Predicted class: 1
Expected class: 1
Match
Output spikes accumulation: 0 9 31 53 40 17 10
Predicted class: 3
Expected class: 3
```

Test	Expected	T	Results (T)	B	Results (B)
0	1	1	Match	1	Match
1	3	3	Match	3	Match
2	2	3	Miss	3	Miss
3	2	2	Match	2	Match
4	6	5	Miss	5	Miss
5	1	1	Match	1	Match
6	1	1	Match	1	Match
7	3	3	Match	3	Match
8	4	4	Match	4	Match

Note: In

simone's nir graph building script, the dt was 1s, but at the last commit of his, he uses dt 1ms. So changed this accordingly in the tau calculation part:

```
tau1 = 1e-4 / (1- beta1)
```

2- Soft Reset

2.1- Mini model

```
cd /home/emre/AllProjects/work_done_presentation/soft-reset/mini_model
```

```
rm example_usage.c lif_neuron_gen.* mini_linear.nir
```

```
python3 torch_to_nir.py
```

(before generating make sure that the scale is 60. This document is prepared with the scale is 60)

```
self.scale_factor = 60.0 # Q15 scaling factor
```

```
python3 nir_to_c_generator_V2_commit_25may.py mini_linear.nir
rm for-soft-mini-model/Core/Src/lif_neuron_gen.c
rm for-soft-mini-model/Core/Inc/lif_neuron_gen.h
cp lif_neuron_gen.c ./for-soft-mini-model/Core/Src/
cp lif_neuron_gen.h ./for-soft-mini-model/Core/Inc/
```

(new terminal)

cd

/home/emre/AllProjects/work_done_presentation/soft-reset/mini_model/for-soft-mini-model
code .

(check if the debug prints there, if not, add this at the end of the

Update_subtract_norecurrent function

```
// DEBUG
char buf[256];
snprintf(buf, sizeof(buf),
         "n:%d | V:%d | w_in:%d | thr:%d | spk:%d | reset:%d |
decay:%d\r\n",
         3,
         (int)neurons[3].membrane_potential,
         (int)weighted_inputs[3],
         (int)thresholds[3],
         (int)output_spikes[3],
         (int)reset_values[3],
         (int)decay_factors[3]);

usart1_print(buf);
```

(delete the debug folder if there is and u copied just now)

(build)

(make sure the board is connected, run and debug)

(Open a new terminal)

tio /dev/ttyACM0

(maybe the name is different check it in the /dev folder)

At this step, this output needs to be seen.

```

Simulation completed. Accuracy: % (0/1)
U are sane, board works!
THE SIMULATION BEGINS...

Timestep=0
n:3 | V:382 | w_in:382 | thr:546 | spk:0 | reset:0 | decay:26214
Timestep=1
n:3 | V:578 | w_in:273 | thr:546 | spk:1 | reset:0 | decay:26214
Timestep=2
n:3 | V:-247 | w_in:-163 | thr:546 | spk:0 | reset:546 | decay:26214
Timestep=3
n:3 | V:-198 | w_in:0 | thr:546 | spk:0 | reset:0 | decay:26214
Timestep=4
n:3 | V:-159 | w_in:0 | thr:546 | spk:0 | reset:0 | decay:26214
Output spikes accumulation: 0 0 0 1 0
Predicted class: 3
Expected class: 0

Miss
Simulation completed. Accuracy: % (0/1)

```

(Now the golden reference)

(new terminal)

cd /home/emre/AllProjects/work_done_presentation/soft-reset/mini_model

python3 golden_ref.py

```

emre@emre-ABRA-A5-V16-6:~/AllProjects/work_done_presentation/soft-reset/mini_model$ python3
golden_ref.py
--- Python Gold Reference (Linear) ---
T=0 | input: 1.0000 | cur: 0.7000 | mem_before: 0.0000 | mem_after: 0.7000 | spk: 0.0000
T=1 | input: 0.0000 | cur: 0.5000 | mem_before: 0.7000 | mem_after: 1.0600 | spk: 1.0000
T=2 | input: 0.0000 | cur: -0.3000 | mem_before: 1.0600 | mem_after: -0.4520 | spk: 0.0000
T=3 | input: 0.0000 | cur: 0.0000 | mem_before: -0.4520 | mem_after: -0.3616 | spk: 0.0000
T=4 | input: 0.0000 | cur: 0.0000 | mem_before: -0.3616 | mem_after: -0.2893 | spk: 0.0000

```

(because of the weight scaling, we need to normalize the board output with this value

(*60/32676))

(60 is the scale, 32676 is the q15 range)

T0, V = 383 = 0,701315348 ~= 0,70

T1, V = 578 = 1,058381909 ~= 1,06

T2, V = -247 = -0,45228431 ~= -0.45

T3, V = -198 = -0,362559893 ~= -0,362

T4, V = -159 = -0,29114658 ~= -0,291

(As we see the values are consistent)

2.2 - Braille7 test

(new terminal)

cd /home/emre/AllProjects/work_done_presentation/soft-reset

rm example_usage.c lif_neuron_gen.* *.nir

python3 torch_to_nir_norec.py

python3 nir_to_c_generator_V2_commit_25may.py manual_braille_model.nir

rm for-soft-reset-norec/Core/Src/lif_neuron_gen.c

for-soft-reset-norec/Core/Inc/lif_neuron_gen.h

cp lif_neuron_gen.c for-soft-reset-norec/Core/Src/

cp lif_neuron_gen.h for-soft-reset-norec/Core/Inc/

(new terminal)
 cd AllProjects/work_done_presentation/soft-reset/for-soft-reset-norec/
 code .
 (delete build content, if its copied recently)
 (build)
 (debug and run)

(Open a new terminal)
 tio /dev/ttyACM0
 (maybe the name is different check it in the /dev folder)
 (it will show 20 different result)

```

Output spikes accumulation: 7 13 3 22 8 13 18
Predicted class: 3
Expected class: 5

Miss
Output spikes accumulation: 4 13 0 14 7 10 10
Predicted class: 3
Expected class: 5

Miss
Output spikes accumulation: 24 8 1 14 14 24 17
Predicted class: 0
Expected class: 0

Match
Output spikes accumulation: 2 25 8 31 13 5 23
Predicted class: 3
Expected class: 2

Miss
Output spikes accumulation: 0 26 8 26 13 3 9
Predicted class: 1
Expected class: 4

Miss
Output spikes accumulation: 0 7 4 39 18 1 9
Predicted class: 3
Expected class: 3

Match
Output spikes accumulation: 0 49 7 15 6 2 14
Predicted class: 1
Expected class: 1

Match
Output spikes accumulation: 0 48 11 12 10 0 13
Predicted class: 1
Expected class: 2

Miss
Simulation completed. Accuracy: % (10/20)

```

```

[ 12] Output spikes accumulation: 13 12 6 26 13 25 25
      Predicted class: 3 | Expected class: 5 X
[ 13] Output spikes accumulation: 6 15 0 20 10 15 17
      Predicted class: 3 | Expected class: 5 X
[ 14] Output spikes accumulation: 34 8 2 14 19 30 24
      Predicted class: 0 | Expected class: 0 ✓
[ 15] Output spikes accumulation: 2 30 11 43 18 7 22
      Predicted class: 3 | Expected class: 2 X
[ 16] Output spikes accumulation: 0 26 10 30 21 3 12
      Predicted class: 3 | Expected class: 4 X
[ 17] Output spikes accumulation: 0 7 2 44 18 1 10
      Predicted class: 3 | Expected class: 3 ✓
[ 18] Output spikes accumulation: 0 49 11 22 11 3 16
      Predicted class: 1 | Expected class: 1 ✓
[ 19] Output spikes accumulation: 0 54 21 14 13 1 14
      Predicted class: 1 | Expected class: 2 X
Overall accuracy: 10/20 (50.00%)

```

(A question? should I create a python script to compare membrane potentials over every timestep for specific neurons, to be sure that its 100% correct like I did in the Convolution part? Or is the mini model test is enough)

(new terminal)
 cd AllProjects/work_done_presentation/soft-reset/
 python3 golden_ref_for_norec.py

3- SCNN

(Use the already trained model output, otherwise need to download the stmnist dataset and then run the training script)

3.1-mini-model

```
cd /home/emre/AllProjects/work_done_presentation/SCNN/mini_model_test
python3 nir_generator_for_mini_model.py
python3 nir_to_c_generator_V2_commit_25may.py mini_conv2d.nir
```

```
rm for_mini_model_stm32h7/Core/Src/lif_neuron_gen.c
for_mini_model_stm32h7/Core/Inc/lif_neuron_gen.h
```

```
cp lif_neuron_gen.c for_mini_model_stm32h7/Core/Src/
cp lif_neuron_gen.h for_mini_model_stm32h7/Core/Inc/
gnome-terminal
```

```
cd for_mini_model_stm32h7/
code .
```

(edit the lif_neuron_gen.c file's base_convolution function, add this to the end to debug)

```
        if (n_idx == 0 && oc == 0 && oh == 0 && ow == 0 ) {
            char buf[200];
            // Use %ld for q31_t (long int) to avoid format
warnings
            // We print the raw integer. 60 = 1.0 in float
terms.
            snprintf(buf, sizeof(buf), "V:%ld = v_shifted:%ld +
acc: %ld - Reset:%ld | threshold: %d S:%d | nindex = %ld | v_prev = %ld
| decay = %ld \r\n",
                    (long)neurons[n_idx].membrane_potential,
v_shifted, acc, reset, neurons[n_idx].threshold,
                    output_spikes[n_idx], n_idx, v_prev,
decay);
            usart1_print(buf);
        }
```

(Delete the debug if the project is copied recently)

(build)

(run and debug)

(Open a new terminal)

`tio /dev/ttyACM0`

(on vscode run/f5)

(should output this)

```
V:436 = v_shifted:0 + acc: 436 - Reset:0 | threshold: 546 S:0 | nindex = 0 | v_prev = 0 | decay = 26214
V:566 = v_shifted:348 + acc: 218 - Reset:0 | threshold: 546 S:1 | nindex = 0 | v_prev = 436 | decay = 26214
V:288 = v_shifted:452 + acc: 382 - Reset:546 | threshold: 546 S:0 | nindex = 0 | v_prev = 566 | decay = 26214
V:230 = v_shifted:230 + acc: 0 - Reset:0 | threshold: 546 S:0 | nindex = 0 | v_prev = 288 | decay = 26214
V:183 = v_shifted:183 + acc: 0 - Reset:0 | threshold: 546 S:0 | nindex = 0 | v_prev = 230 | decay = 26214
[00] T= 5 GT=0 Pred=0 PASS spikes: 1 1 1 1 1 0 0
```

(Open a new terminal)

`cd AllProjects/work_done_presentation/SCNN/mini_model_test/`

`python3 mini_model_reference.py`

```
emre@emre-ABRA-A5-V16-6:~/AllProjects/work_done_presentation/SCNN/mini_model_test$ python3 mini_model_reference.py
--- Python Gold Reference (Conv2d) ---
T=0 | cur: 0.8000 | mem_before: 0.0000 | mem_after: 0.8000 | spk: 0.0
T=1 | cur: 0.4000 | mem_before: 0.8000 | mem_after: 1.0400 | spk: 1.0
T=2 | cur: 0.7000 | mem_before: 1.0400 | mem_after: 0.5320 | spk: 0.0
T=3 | cur: 0.0000 | mem_before: 0.5320 | mem_after: 0.4256 | spk: 0.0
T=4 | cur: 0.0000 | mem_before: 0.4256 | mem_after: 0.3405 | spk: 0.0
```

(Normalized, values with $*60/32767$)

Timestep 0, $V = 436 = 0,798339844 \approx 0,8$

Timestep 1, $V = 566 = 1,036376953 \approx 1,04$

Timestep 2, $V = 288 = 0,52734375 \approx 0,53$

Timestep 3, $V = 230 = 0,421142578 \approx 0,42$

Timestep 4, $V = 183 = 0,335083008 \approx 0,34$

3.2-st-mnist

`cd AllProjects/work_done_presentation/SCNN/`

`python3 SNNtoNIR.py`

`python3 nir_to_c_generator_V2_commit_25may.py stmnist_with_reset.nir`

`rm for_SCNN_stm32h7/Core/Src/lif_neuron_gen.c`

`for_SCNN_stm32h7/Core/Inc/lif_neuron_gen.h`

`cp lif_neuron_gen.c for_SCNN_stm32h7/Core/Src/`

`cp lif_neuron_gen.h for_SCNN_stm32h7/Core/Inc/`

`gnome-terminal`

`cd for_SCNN_stm32h7/`

`code .`

(Inside of the lif_neuron_gen.c, add the attribute section before the conv2_weights_vector to store it in the QSPI. If this is lacking, it might be because the nir_to_c generator does not include this because this is board specific)

```
// Layer 2 conv weights - Conv2d (64x32x3x3)
// Stored in OUT_CH-MAJOR order: [oc][ic][kh][kw]
__attribute__((section(".ext_weights")))
static const float conv2_weights_vector[18432] = {
```

(Delete the debug if the project is copied recently)

(build)

(after the build is done, we need to write this weights and the test data to the board's qspi manually)

(new terminal)

```
cd AllProjects/work_done_presentation/SCNN/for_SCNN_stm32h7/build/Debug/
```

```
ls
```

(there needs to be the .elf file)

```
arm-none-eabi-objcopy -O binary --only-section=.ext_weights for_SCNN_stm32h7.elf
weights_and_tests.bin
```

```
arm-none-eabi-objcopy --remove-section=.ext_weights for_SCNN_stm32h7.elf
for_SCNN_stm32h7_debug.elf
```

```
~/STMicroelectronics/STM32Cube/STM32CubeProgrammer/bin/STM32_Programmer_CLI
-c port=SWD -el
```

```
~/STMicroelectronics/STM32Cube/STM32CubeProgrammer/bin/ExternalLoader/MT25TL01
G_STM32H750B-DISCO.stldr -d weights_and_tests.bin 0x90000000 -v
```

(If not, in VSCODE go to debug and run section on the left. Press create a launch.json)

(Inside change this dynamic elf name finder with a static one like this to ours.

```
"imageFileName": "${workspaceFolder}/build/Debug/for_SCNN_stm32h7_debug.elf")
```

(run and debug)

(Open a new terminal)

```
tio /dev/ttyACM0
```

(on vscode run/f5)


```

[180] T= 87 GT=8 Pred=8 PASS spikes: 1 7 11 17 1 53 12 9 78 12
[181] T= 75 GT=4 Pred=4 PASS spikes: 5 12 22 33 54 25 3 9 0 2
[182] T= 89 GT=3 Pred=3 PASS spikes: 22 14 10 62 12 17 21 46 15 45
[183] T= 71 GT=6 Pred=6 PASS spikes: 17 10 6 12 16 22 46 7 10 24
[184] T= 58 GT=4 Pred=5 FAIL spikes: 3 7 16 31 30 34 12 1 6 2
[185] T= 39 GT=4 Pred=4 PASS spikes: 8 7 8 1 11 0 9 1 6 1
[186] T= 79 GT=0 Pred=0 PASS spikes: 47 20 3 44 2 10 1 9 15 12
[187] T= 71 GT=6 Pred=6 PASS spikes: 7 28 10 28 14 4 47 10 14 0
[188] T= 71 GT=0 Pred=0 PASS spikes: 54 4 9 17 0 42 11 19 19 10
[189] T= 68 GT=9 Pred=9 PASS spikes: 18 4 9 0 0 1 7 14 0 63
[190] T= 71 GT=6 Pred=6 PASS spikes: 30 3 11 22 7 1 51 14 7 21
[191] T= 59 GT=5 Pred=5 PASS spikes: 5 0 2 4 9 47 1 17 7 7
[192] T= 43 GT=2 Pred=7 FAIL spikes: 15 0 32 23 5 10 3 42 1 0
[193] T= 89 GT=4 Pred=4 PASS spikes: 2 31 26 39 58 25 21 29 8 27
[194] T= 33 GT=0 Pred=0 PASS spikes: 24 2 2 12 0 12 2 8 7 10
[195] T= 48 GT=9 Pred=9 PASS spikes: 30 0 0 3 4 24 0 4 4 32
[196] T= 45 GT=8 Pred=8 PASS spikes: 6 1 2 9 1 14 0 7 19 12
[197] T= 34 GT=5 Pred=5 PASS spikes: 3 3 22 13 0 29 4 4 2 0
[198] T= 47 GT=0 Pred=0 PASS spikes: 22 1 8 10 0 13 5 7 3 15
[199] T= 60 GT=9 Pred=9 PASS spikes: 18 5 12 9 3 6 13 8 2 44

```

Accuracy: 147/200 (73.50%)

```

[147] T= 70 GT=4 Pred=4 PASS spikes: 2 0 2 67 23 1 2 11 12
[148] T= 40 GT=0 Pred=0 PASS spikes: 4 0 0 27 0 23 0 0 4 12
[149] T= 28 GT=9 Pred=9 PASS spikes: 7 3 2 12 0 7 13 10 20 21
[150] T= 52 GT=9 Pred=9 FAIL spikes: 19 4 0 30 0 5 0 0 5 20
[151] T= 18 GT=3 Pred=3 PASS spikes: 0 0 0 10 0 3 7 0 1 1
[152] T= 47 GT=3 Pred=0 FAIL spikes: 33 0 6 33 2 11 0 17 4 5
[153] T= 50 GT=0 Pred=0 PASS spikes: 3 0 0 3 4 18 23 0 11 14
[154] T= 51 GT=9 Pred=9 PASS spikes: 0 0 0 20 0 19 0 0 15 37
[155] T= 57 GT=2 Pred=2 PASS spikes: 10 0 39 0 0 21 2 20 6 5
[156] T= 20 GT=0 Pred=0 PASS spikes: 20 2 10 17 0 20 0 0 0 3
[157] T= 47 GT=4 Pred=4 PASS spikes: 0 1 14 0 20 1 1 14 7 4
[158] T= 57 GT=0 Pred=0 FAIL spikes: 29 11 0 12 2 30 32 0 6 4
[159] T= 88 GT=8 Pred=8 PASS spikes: 26 17 7 53 10 13 14 17 48 15
[160] T= 42 GT=2 Pred=2 PASS spikes: 1 0 37 20 0 1 3 14 7 6
[161] T= 58 GT=9 Pred=2 FAIL spikes: 19 1 39 9 0 24 14 4 8 34
[162] T= 38 GT=7 Pred=7 PASS spikes: 4 0 12 28 0 4 0 38 10 0
[163] T= 40 GT=5 Pred=3 FAIL spikes: 33 1 3 0 7 0 43 0 4 5 10
[164] T= 40 GT=0 Pred=0 FAIL spikes: 29 0 5 7 0 0 20 3 2 1
[165] T= 38 GT=9 Pred=5 FAIL spikes: 2 0 23 5 2 24 0 0 0 19
[166] T= 32 GT=2 Pred=2 PASS spikes: 10 0 32 17 1 3 0 17 1 0
[167] T= 79 GT=4 Pred=4 PASS spikes: 6 6 7 40 60 7 6 9 5 5
[168] T= 20 GT=9 Pred=5 FAIL spikes: 5 0 1 18 1 23 1 4 3 7
[169] T= 99 GT=0 Pred=0 PASS spikes: 17 13 21 1 17 29 60 23 24 17
[170] T= 44 GT=3 Pred=3 PASS spikes: 38 9 11 39 0 0 0 6 2 0
[171] T= 37 GT=0 Pred=0 FAIL spikes: 24 9 4 3 0 8 11 0 0 2
[172] T= 34 GT=7 Pred=7 PASS spikes: 12 1 8 15 0 12 0 34 13 0
[173] T= 89 GT=0 Pred=0 PASS spikes: 67 3 12 15 26 18 33 9 10 8
[174] T= 94 GT=1 Pred=1 PASS spikes: 20 64 17 0 23 20 13 15 19 14
[175] T= 55 GT=2 Pred=7 FAIL spikes: 16 0 17 30 2 13 11 39 0 1
[176] T= 84 GT=5 Pred=5 PASS spikes: 10 6 3 32 19 84 23 12 4 6
[177] T= 19 GT=7 Pred=9 FAIL spikes: 3 9 1 0 8 2 0 0 12
[178] T= 19 GT=0 Pred=0 FAIL spikes: 1 0 2 1 14 4 2 3 2 0
[179] T= 74 GT=1 Pred=1 PASS spikes: 10 66 17 8 17 17 12 21 18 5
[180] T= 87 GT=8 Pred=8 PASS spikes: 1 7 11 17 1 53 12 9 78 12
[181] T= 75 GT=4 Pred=4 PASS spikes: 5 12 22 33 54 25 3 9 0 2
[182] T= 89 GT=3 Pred=3 PASS spikes: 22 14 10 62 12 17 21 46 15 45
[183] T= 71 GT=6 Pred=6 PASS spikes: 17 10 6 12 16 22 46 7 10 24
[184] T= 58 GT=4 Pred=5 FAIL spikes: 3 7 16 31 30 34 12 1 6 2
[185] T= 39 GT=4 Pred=4 PASS spikes: 8 7 8 1 11 0 9 1 6 1
[186] T= 79 GT=0 Pred=0 PASS spikes: 47 20 3 44 2 10 1 9 15 12
[187] T= 71 GT=6 Pred=6 PASS spikes: 7 28 10 28 14 4 47 10 14 0
[188] T= 71 GT=0 Pred=0 PASS spikes: 54 4 9 17 0 42 11 19 19 10
[189] T= 68 GT=9 Pred=9 PASS spikes: 18 4 9 0 0 1 7 14 0 63
[190] T= 71 GT=6 Pred=6 PASS spikes: 30 3 11 22 7 1 51 14 7 21
[191] T= 59 GT=5 Pred=5 PASS spikes: 5 0 2 4 9 47 1 17 7 7
[192] T= 43 GT=2 Pred=7 FAIL spikes: 15 0 32 23 5 10 3 42 1 0
[193] T= 89 GT=4 Pred=4 PASS spikes: 2 31 26 39 58 25 21 29 8 27
[194] T= 33 GT=0 Pred=0 PASS spikes: 24 2 2 12 0 12 2 8 7 10
[195] T= 48 GT=9 Pred=9 PASS spikes: 30 0 0 3 4 24 0 4 4 32
[196] T= 45 GT=8 Pred=8 PASS spikes: 6 1 2 9 1 14 0 7 19 12
[197] T= 34 GT=5 Pred=5 PASS spikes: 3 3 22 13 0 29 4 4 2 0
[198] T= 47 GT=0 Pred=0 PASS spikes: 22 1 8 10 0 13 5 7 3 15
[199] T= 60 GT=9 Pred=9 PASS spikes: 18 5 12 9 3 6 13 8 2 44

```

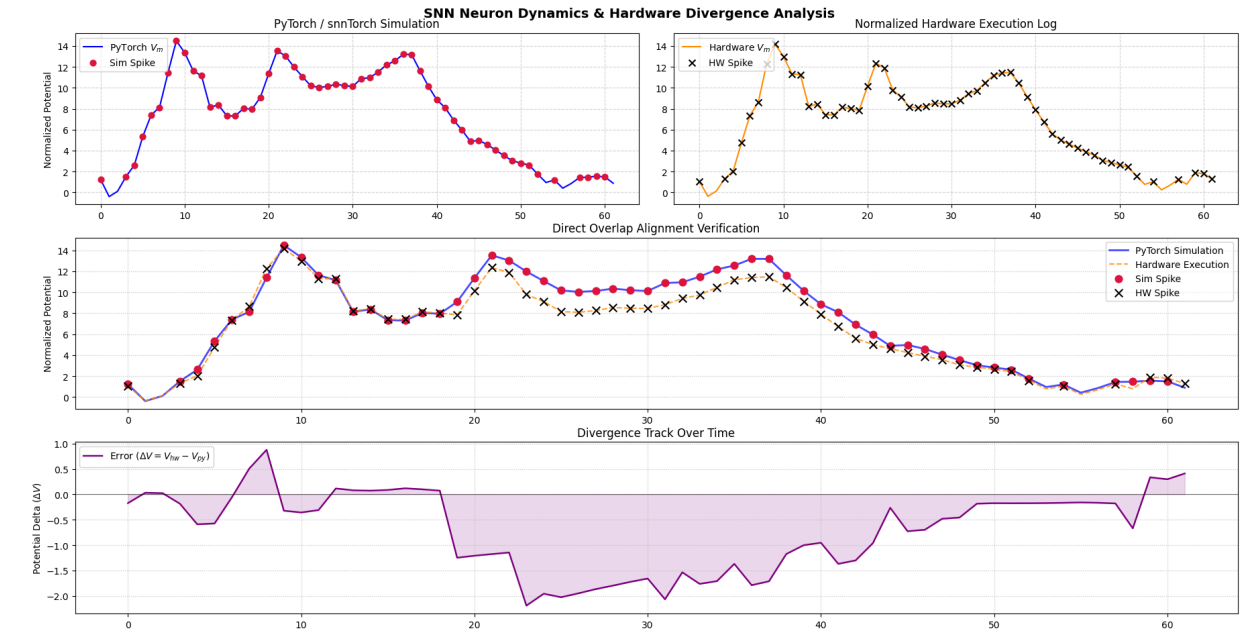
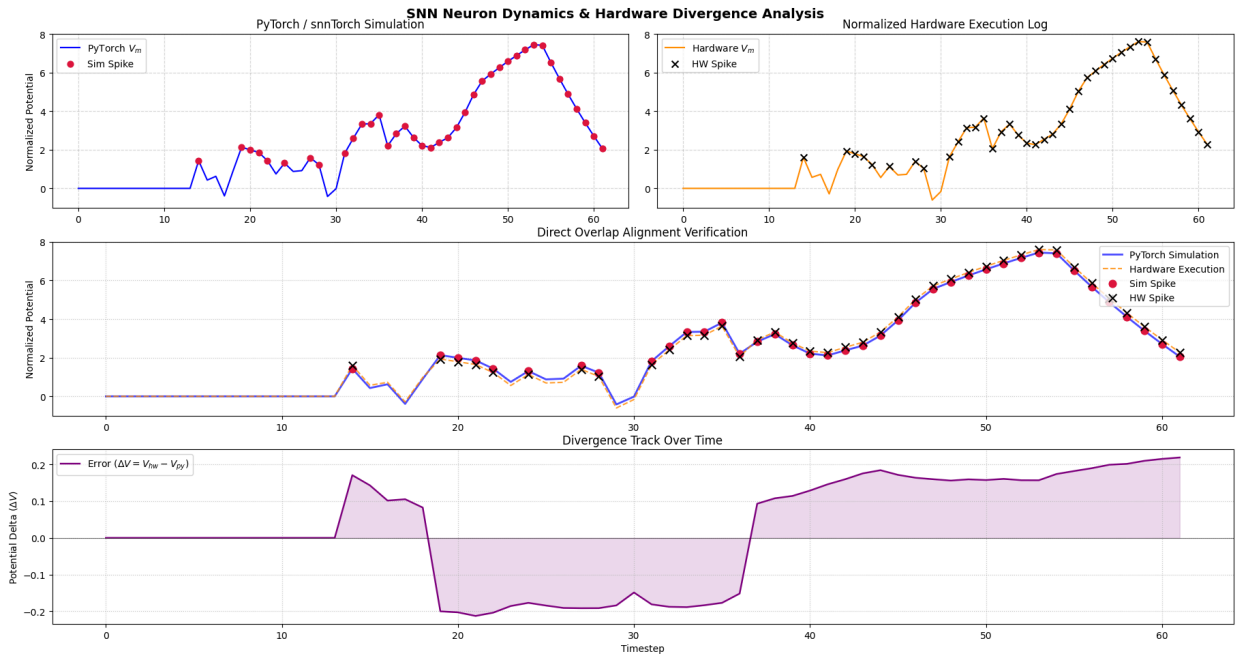
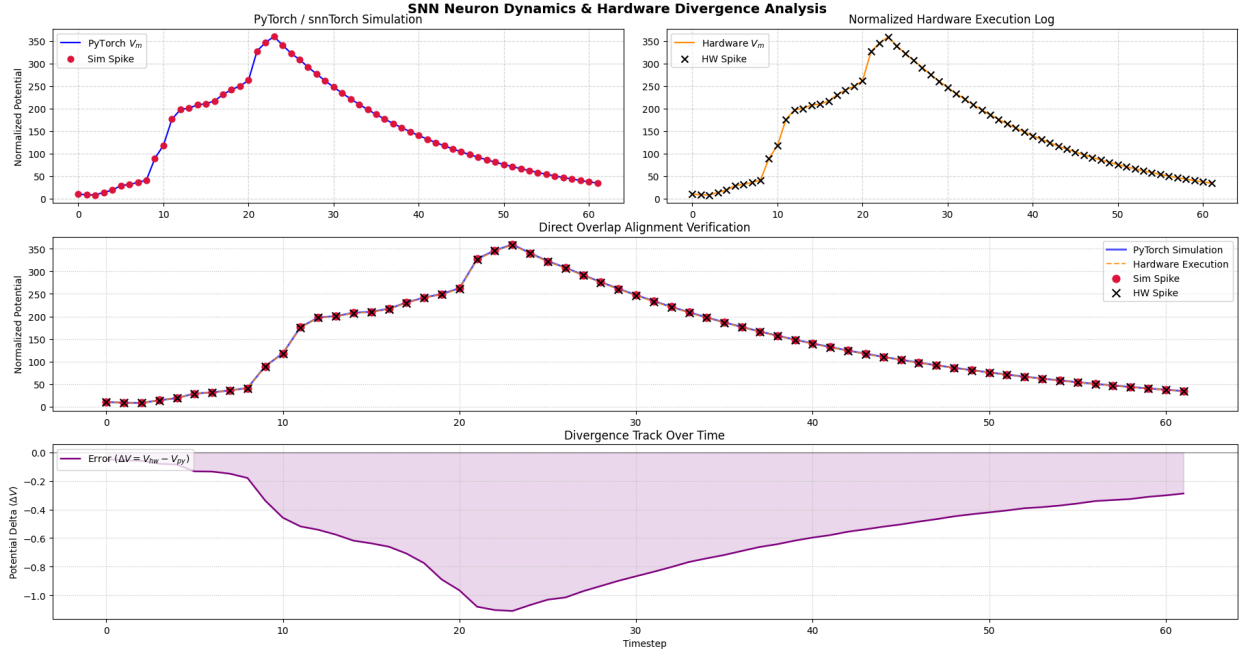
Accuracy: 147/200 (73.50%) Final Accuracy: 150/200, 75.00%

(To test its full correctness, python scripts are written)

(Can check this folder to see its details)

cd /home/emre/AllProjects/work_done_presentation/SCNN/debug_and_plot

(Here are the results)



- a. "It means that they are still not capable of implementing Convolutional Spiking Neural Networks and focus on Feedforward SNNs so for now they beat all the Python implementations in speed but might not beat the C++ SNNs (there is a lack of benchmarks comparing those C SNNs to existing C++ SNNs - my peer review for ya)."

- i. From

- https://www.linkedin.com/posts/now-there-are-snns-in-the-c-programming-language-ugcPost-7470181759704514561-tn_K/?utm_source=social_share_send&utm_medium=android_app&rcm=ACoAABLnSd8BSr1MXeS8bgboP56aKN0XLH47iO4&utm_campaign=copy_link

- ii. Professor sent, maybe I can compare my work with C++ version.